
VLM-Verified Counterfactual Hindsight for Sparse-Reward Manipulation

Daniel Grant Parshawn Gerafian Matei Gardea
Department of Electrical Engineering and Computer Sciences
University of California, Berkeley
{dgrant,pgerafian,mgardea}@berkeley.edu

Abstract

Sparse-reward manipulation is a credit-assignment problem: temporal-difference signal decays exponentially away from the one bit of terminal reward, so the bulk of every trajectory is effectively unprioritized. We make two contributions. First, we recast VLM-guided experience replay as *importance-sampled posterior reweighting*: a frozen Vision-Language-Model emits an approximate posterior $q_\phi(t^* | \tau)$ over which timestep caused an episode’s outcome, and Semantic PER multiplies the standard TD-error priority by this posterior. Within this framing the multiplicative form is the unique proposal modification that preserves PER’s importance-sampling correction; the additive mixture used by the concurrent VLM-RB [25] implicitly retargets the loss. Second, we introduce **VLM-Verified Counterfactual Hindsight**, which closes the dominant failure mode of language-only counterfactual goal proposals: rather than asking the VLM for a hindsight goal (which teleport-collapses in 100% of Claude Opus 4.7 calls on FetchPush), we ask for a corrective action sequence and execute it in a fork of the training simulator from the failure state. Only simulator-verified relabels enter the buffer, with confidence 1.0 and zero modeling error. On Gymnasium-Robotics Fetch with SAC, a head-to-head GPT-4o vs. Opus 4.7 vs. Sonnet 4.5 prompt sweep on 6 synthetic and 10 real failed episodes identifies (GPT-4o, achieved_goal + 5 cm gate) as strictly dominant (0% teleport-collapse, 0.79 plausibility, 0.83 goal-progress); a 12-episode cross-task pilot achieves 12/12 task-relevant annotations across Push, PickAndPlace, and Slide using a single prompt template, evidence that a foundation-VLM credit-assignment oracle transfers where a learned priority head would not.

1 Introduction

Sparse-reward manipulation in environments such as Gymnasium-Robotics Fetch is defined by long horizons (50 steps), one bit of terminal reward, and a goal-achieved threshold of 5 cm. Standard prioritized experience replay (PER) [24] sharpens the sampling distribution toward transitions with large TD-error, but TD-error itself decays exponentially away from the goal-achievement timestep. Hindsight Experience Replay (HER) [1] partially repairs this by relabeling failures as successes for an alternative achieved goal, but the relabel target is constrained to the agent’s actually-realized trajectory.

Recent work has explored using foundation Vision-Language Models (VLMs) as “oracles” inside the RL loop, both for reward shaping [22, 14, 27] and, very recently, for replay-buffer prioritization [25]. We argue these uses are best framed not as engineering recipes but as *importance-sampled stochastic optimization with a foundation-model proposal distribution*: the VLM is a frozen approximation to the (intractable) posterior $p(t^* | \tau)$ over which transitions caused an episode’s outcome.

This paper makes four contributions.

(1) An IS-posterior framing of VLM-guided replay (Section 3). We re-derive prioritized DQN [24], retrace [18], V-trace [7], and Sharony et al.’s VLM-RB [25] inside a single off-policy-correction taxonomy, locating our scheme as *proposal-shaping* with an exogenous foundation-model credit-assignment oracle. The multiplicative form of our update is the *unique* proposal modification (up to trajectory-conditional normalization) that preserves the standard PER importance-sampling correction; the additive mixture used by VLM-RB [25] implicitly retargets the loss to a λ -weighted objective.

(2) Semantic PER with a failure-direction signal (Section 4.1). Sharony et al. score 32-frame clips for goal-satisfaction; we ask the VLM the strictly more informative question—*which single timestep made this trajectory fail?*—and multiply the standard PER priority by a bounded boost over a window around that timestep. The scheme degenerates gracefully to PER when the VLM is uncommitted ($w_{\max} = 1$), needs no λ -warm-up, and admits an unbiased IS-corrected interpretation. Crucially, unlike VLM-as-reward approaches [22, 30, 20] that splice the VLM signal into the TD target, our scheme leaves the target untouched: the env’s true sparse reward is recomputed on every sampled transition. VLM miscalibration costs variance in the sampling distribution, not bias in the value function.

(3) VLM-Verified Counterfactual Hindsight (Section 4.3). A direct prompt for an alternative *achieved goal* elicits a teleport-collapse failure mode in 100% of Claude Opus 4.7 calls on FetchPush: the VLM returns `corrective_position = desired_goal`, a degenerate zero-information output. We close this loophole *by construction* by asking instead for a corrective *action sequence* and executing it in a fork of the training simulator from the failure-timestep state. A four-vector action cannot teleport a 50 g block by 50 cm under MuJoCo dynamics; the failure mode is structurally inexpressible. The mechanism instantiates the *generator-verifier* paradigm [31] in robot RL—the VLM generates, the simulator verifies—and only trajectories whose sparse reward fires under the unmodified environment dynamics enter the buffer, with confidence 1.0 and zero modeling error.

(4) Empirical analysis: prompt design, VLM-family robustness, and cross-task transfer (Sections 5.3–5.5). A head-to-head GPT-4o vs. Claude Opus 4.7 sweep on six synthetic Fetch failures identifies a strictly-dominating configuration: (GPT-4o, `achieved_goal`) eliminates teleport-collapse (0/6 vs. Opus’s 4/4), ties for highest plausibility (0.79), and trails only Opus’s degenerate goal-copying on goal-progress (0.83 vs. 0.97). A second prompt sweep on ten *real* SAC-failure episodes confirms the prompt-architectural fix transfers to a third VLM family (Sonnet 4.5: 75% $\rightarrow$ 0% teleport on PickAndPlace). A 12-episode cross-task pilot achieves 12/12 task-relevant annotations on Push/PickAndPlace/Slide with a single prompt template, evidence that a foundation-VLM credit-assignment oracle generalizes across structurally-different manipulation tasks where any learned priority head would require per-task fitting.

Section 5 reports SAC+HER baselines on the three Fetch environments, the prompt-design analysis, cross-task transfer evidence, real-data validation, and the current status of two in-flight experiments (HER baselines and a kill-test of the verified-counterfactual mechanism). Section 6 discusses a pipeline-bug discovery that re-frames our headline ablation, small- n caveats, and limitations.

2 Related Work

HER and goal-conditioned RL. Hindsight Experience Replay [1] is the dominant solution to sparse-reward goal-conditioned manipulation; the future relabel strategy with $k = 4$ remains a strong baseline a decade later. Recent sample-efficiency improvements include single-step relabeling (Next-Future) [19], contact-energy prioritization (CEBP) [23], hindsight regularization (GCHR) [15], and null-counterfactual interaction inference (HInt/NCII) [4]. HInt is the closest in spirit to our verified counterfactuals: it asks “would the target object’s dynamics change if the cause object were removed?” using a learned dynamics model. We use a frozen VLM rather than a learned model and verify in the real simulator.

Prioritized replay and off-policy correction. Prioritized DQN [24] samples in proportion to $(|\delta| + \varepsilon)^\alpha$ with an annealed importance-sampling weight $w_{\text{IS}} = (|\mathcal{D}| \cdot \mu_P)^{-\beta}$. Retrace [18] and V-trace [7] extend off-policy correction to multi-step returns and distributed actor-learner setups, respectively, via clipped per-step IS ratios. Ma et al. [16] address PER staleness in the LLM/VLM regime via age-decay weights. Our Semantic PER sits inside this taxonomy as a *proposal-shaping*

Table 1: VLM-Guided Experience Replay (Sharony et al. 2026) vs. this paper.

Axis	VLM-RB [25]	This paper
Signal direction	Goal satisfaction (success)	Failure-causing transitions
Granularity	32-frame sub-trajectory clip	Single timestep $\pm$ window
Blending with TD-error	Mixture-with-uniform; $\lambda_t : 0 \rightarrow 0.5$ warm-up	Multiplicative; degenerates to PER
Headroom analysis	—	Heuristic privileged-state oracle
Counterfactual relabel	—	VLM-proposed action + sim-fork verification
Cross-task evidence	Per-benchmark tuning of λ , TD multiplier	One prompt template across 3 Fetch envs
Benchmarks	MiniGrid, OGBench (DQN/IQN/SAC/TD3)	Gymnasium-Robotics Fetch (SAC)

scheme whose modifier (the VLM posterior) is exogenous and trajectory-level rather than policy-derived; see Table 2.

Foundation models in RL. VLMs have been used as reward sources [22, 27, 28], affordance-guided shaping signals [14], promptable feature encoders [3], counterfactual data labelers [9], token-level counterfactual critics [8], hindsight rewriters for LM agents [12, 5], and online per-timestep reward models for manipulation [30, 20]. Duan et al. [6] fine-tune a VLM (AHA) on synthetic robotic failures and produce natural-language failure explanations that downstream-refine LLM-generated reward code (Eureka pipeline); our system differs in three orthogonal ways. (i) AHA *describes* a failure; we *localize* it to a single timestep usable as a credit-assignment signal. (ii) AHA proposes no corrective actions; we do, and gate them on simulator verification. (iii) AHA’s RL integration is reward-refinement (modifies the TD target); ours is replay-prioritization (modifies only the sampling distribution). An AHA-fine-tuned VLM could plausibly replace zero-shot GPT-4o inside our protocol. Foundation-model credit assignment is itself a young thread: Khandoga et al. [13] mask reasoning spans in an LLM policy to identify causally-influential tokens, in the lineage of Mesnard et al. [17] and the credit-assignment survey by Pignatelli et al. [21].

Generator–verifier and verified counterfactual reasoning. Our verification protocol (Section 4.3) sits inside the *generator–verifier* paradigm now popular in LLM reasoning: the generator emits candidates, a verifier (learned or symbolic) accepts the correct ones [31]. We instantiate this in robotics with a foundation-VLM generator and a *symbolic* verifier—the unmodified training simulator—whose decisions are correct by construction. The most related prior work in robotics is the line on counterfactual data augmentation for goal-conditioned RL [26, 4, 9]: each uses a learned proxy (causal-action-influence, learned dynamics, VLM plausibility) to produce additional trajectories. We use a ground-truth proxy (the simulator), which carries zero modeling error, at the cost of additional sim-fork compute.

Differentiation from VLM-RB. The concurrent (Feb 2026) VLM-RB [25] is the closest published work: a frozen VLM scores sub-trajectory clips for prioritized replay on MiniGrid and OGBench, reporting 11–52% higher success and 19–45% better sample efficiency. The two systems differ along five orthogonal axes summarized in Table 1, with two methodological cores worth emphasizing. **First**, VLM-RB asks “is this sub-trajectory good?” (success-direction scoring) and mixes the answer *additively* with uniform sampling $\mu = \lambda\mu_{\text{VLM}} + (1 - \lambda)\mu_U$; we ask the strictly more informative “which single timestep was outcome-causing?” and combine the answer *multiplicatively* with TD-error $\mu \propto \mu_P \cdot w_{\text{sem}}$. The multiplicative form is what preserves PER’s importance-sampling-correction interpretation (Section 3); the additive mixture implicitly retargets the loss to a λ -weighted objective that VLM-RB does not correct for. **Second**, we ship a verified-counterfactual-hindsight mechanism (Section 4.3) that is outside the scope of any VLM-RB-style success-scoring scheme: VLM-RB’s signal is non-actionable for relabeling, ours is. To our knowledge this paper is the first to (a) use a VLM-localized failure timestep as a credit-assignment signal and (b) verify counterfactual relabels in the same simulator on which the policy trains.

3 Theoretical Motivation: VLM-Guided PER as Posterior Reweighting

Setup. Let $\tau = (s_0, a_0, r_0, \dots, s_T)$ be an episode trajectory and $i \in \{0, \dots, T-1\}$ a transition index. Off-policy value learning solves a regression problem on a replay-sampling distribution $\mu(i)$ over a buffer $\mathcal{D}_B$:

$$\mathcal{L}(\theta) = \mathbb{E}_{i \sim \mu} [\ell(\delta_i(\theta))], \quad \delta_i = r_i + \gamma Q(s_{i+1}, \pi(s_{i+1})) - Q(s_i, a_i). \quad (1)$$

Uniform replay $\mu_U(i) = 1/|\mathcal{D}_B|$ is unbiased but variance-heavy. Prioritized replay [24] uses $\mu_P(i) \propto (|\delta_i| + \varepsilon)^\alpha$ and recovers (an annealed estimator of) the uniform objective by reweighting each gradient by the IS correction

$$w_{\text{IS}}(i) = (|\mathcal{D}_B| \cdot \mu_P(i))^{-\beta}, \quad \beta: \beta_0 \rightarrow 1. \quad (2)$$

This is an instance of self-normalized importance sampling: the proposal μ_P concentrates on high-error transitions; the IS-weight returns the target to $\mathbb{E}_U[\ell]$ in expectation.

The VLM as a posterior. A frozen VLM that answers “which timestep caused this trajectory’s outcome?” approximates the posterior $p(t^* | \tau) \propto p(\tau | t^*)p(t^*)$ where t^* is the (latent) outcome-causing timestep. Write $q_\phi(t^* | \tau)$ for the VLM’s approximation; in our implementation it is the argmax of a chain-of-thought prompt, but it can equivalently be elicited as a soft distribution. We define a per-transition semantic boost

$$w_{\text{sem}}(i; \tau) := \mathbb{E}_{t^* \sim q_\phi(\cdot | \tau)} [K_W(i; t^*)], \quad K_W(i; t^*) = 1 + (w_{\text{max}} - 1) \mathbb{I}[|i - t^*| \leq W], \quad (3)$$

with window half-width W and bounded boost $w_{\text{max}} \geq 1$. The semantic-PER proposal is the product

$$\boxed{\mu_{\text{Sem}}(i) \propto \mu_P(i) \cdot w_{\text{sem}}(i; \tau_i)} \quad (4)$$

Two factors drive priority: $\mu_P(i)$ captures *learning progress* (how much the value head would reduce its loss by training on i); w_{sem} captures *causal influence* (how much posterior mass the VLM places on i being outcome-causing). Standard PER uses only the first; uniform replay uses neither; Semantic PER multiplies the two. When q_ϕ is near-uniform, $w_{\text{sem}} \approx 1$ and we degenerate to PER.

Off-policy correction taxonomy. Eq. (4) sits inside a well-charted off-policy-correction taxonomy (Table 2). In our current implementation we use the PER IS-weight $w_{\text{IS}, P}$ rather than the strictly-correct $w_{\text{IS}, \text{Sem}} = (|\mathcal{D}_B| \mu_{\text{Sem}})^{-\beta}$. This is a conservative under-correction (semantic-boosted transitions are sampled more than their IS weight reflects) and is mathematically analogous to V-trace’s ratio truncation: a controlled bias for variance reduction.

Why multiplicative? Uniqueness, additive mixtures, and what this buys us. Two predictions follow from Eq. (4) that distinguish our scheme from neighboring options. **(P1) Uniqueness of multiplicative.** The PER IS-correction $w_{\text{IS}}(i) = (|\mathcal{D}_B| \mu(i))^{-\beta}$ recovers the uniform-replay objective as $\beta \rightarrow 1$. Any proposal modification $\mu'(i) = f(\mu_P(i), q_\phi(i; \tau))$ that wants the same correction interpretation must satisfy $\mu'(i) \propto \mu_P(i) \cdot g(q_\phi(i; \tau))$ for some trajectory-conditional g : the IS denominator must factor so that the $\beta \rightarrow 1$ limit returns the uniform-replay expectation. An additive mixture $\lambda \mu_q + (1 - \lambda) \mu_P$ [25] does *not* satisfy this: under $w_{\text{IS}} = (|\mathcal{D}_B| (\lambda \mu_q + (1 - \lambda) \mu_P))^{-\beta}$ the $\beta \rightarrow 1$ limit is the λ -weighted objective $\lambda \mathbb{E}_{\mu_q}[\ell] + (1 - \lambda) \mathbb{E}_{\mu_P}[\ell]$, *not* $\mathbb{E}_U[\ell]$, unless λ is itself folded into the correction. The multiplicative form is, up to normalization, the unique proposal modification that keeps PER’s interpretation. **(P2) Replay-shaping vs. reward-shaping.** A natural alternative use of the VLM is to splice it into the TD target as a per-timestep reward [30, 20, 22]; that propagates VLM calibration error directly into Q as bias. Our scheme keeps the env’s true sparse reward intact in the TD target and uses the VLM only to modify μ ; VLM miscalibration shows up only as variance in the sampling distribution, corrected by w_{IS} . The framing forces a clean prediction: *semantic-PER should track standard PER closely when the VLM is poorly calibrated, and pull ahead when the VLM is reliable; neither curve should diverge*—a prediction reward-shaping cannot make.

Connection to causal credit assignment. The credit-assignment survey of Pignatelli et al. [21] categorizes methods by how they identify outcome-influential actions. Endogenous oracles include TD-error (PER), return-decomposition (RUDDER [2], HCA [11]), and counterfactual policy-gradient baselines (CCA-PG [17]). Khandoga et al. [13] extends the counterfactual idea to LLM tokens

Table 2: Off-policy correction taxonomy. Semantic PER (this paper) is a proposal-shaping scheme whose modifier is exogenous and trajectory-level rather than policy-derived.

Method	Proposal μ	Correction	Bias source
Retrace [18]	Behavior policy μ_b	Clipped per-step IS c_s	IS-ratio truncation
V-trace [7]	Behavior policy μ_b	Doubly-clipped $\bar{\rho}, \bar{c}$	Both-ratio truncation
Prioritized DQN [24]	$ \delta ^\alpha$	$(\mathcal{D} \mu_P)^{-\beta}$	Anneal of $\beta < 1$
VLM-RB [25]	$\lambda\mu_{\text{VLM}}^P + (1-\lambda)\mu_U$	none published	Mixture objective $\neq$ PER
Semantic PER (ours)	$\mu_P \cdot w_{\text{sem}}$ (Eq. 4)	$w_{\text{IS},P}$ (under-corrected)	q_ϕ miscalibration + under-correction

by masking reasoning spans. *Our semantic-PER signal introduces a new category: exogenous foundation-model credit-assignment oracles*, where the oracle is pre-trained on a different distribution (web video, image-text) and queried zero-shot at the trajectory level for causal localization. This admits a strictly stronger framing of our contribution than “VLM in replay”: we use foundation-model priors as credit-assignment oracles, with verifiable counterfactual hindsight as the acceptance gate.

4 Method

4.1 Semantic PER with the Failure-Direction Signal

At the end of each failed training episode, we render $K = 5$ uniformly-sampled keyframes plus a task description, query a frozen VLM for the `failure_frame_index` $\in \{0, \dots, K-1\}$, and map this back to a timestep $t_{\text{fail}} \in \{0, \dots, T-1\}$. For each in-buffer slot i of that episode we apply a multiplicative weight

$$w_i = \begin{cases} w_{\text{max}} & |i - t_{\text{fail}}| \leq W, \\ 1 & \text{otherwise} \end{cases}$$

with $w_{\text{max}} = 10$ and $W = 5$ steps as defaults. The priority becomes $p_i = (|\delta_i| + \varepsilon)^\alpha \cdot w_i$ (with $\alpha = 0.6$); we keep the raw TD priority in a sibling array so re-blending after weight changes is exact (no stale sum-tree leaves).

The *bidirectional* variant (used as an upper-bound experimental control) adds a *success* window centered on `argmax-window` distance-reduction; this is Sharony et al.’s direction but per-timestep. Slots inside both windows receive $w_{\text{max}} \times w_{\text{max}} = 100\times$ the baseline TD priority. The contact-aware “Oracle v3” heuristic, used as a privileged-state upper envelope, walks the trajectory for contact-loss events ($\|\text{ee} - \text{obj}\| < 0.05$ m to far-from-contact) with the object still > 0.10 m from the goal.

4.2 Counterfactual Prompting and the Teleport-Collapse Failure Mode

For verified-counterfactual hindsight we additionally need a corrective signal. We define four prompt variants: (NARRATIVE) a free-form English diagnosis; (ACTION) a corrective 4-vector $(\Delta x, \Delta y, \Delta z, \text{grip})$; (ACHIEVED_GOAL) a hindsight achieved_goal $\in \mathbb{R}^3$; (ALL) a unified JSON with all three fields.

Empirically, Claude Opus 4.7 teleport-collapses on ACHIEVED_GOAL in **100% of FetchPush episodes (4/4)** by outputting `corrective_position = desired_goal` (Section 5.3). GPT-4o eliminates this on ACHIEVED_GOAL (0/6 collapse) but partially inherits it on ALL (4/6 collapse), suggesting the failure mode is *prompt-architectural* rather than model-specific. We adopt (GPT-4o, achieved_goal) as our production configuration, with a `reject_teleport_radius = 0.05` m gate as a belt-and-suspenders fallback.

4.3 VLM-Verified Counterfactual Hindsight

The teleport-collapse failure mode (Section 4.2) is the dominant pathology of VLM-as-counterfactual-oracle in our domain. C1’s adapter band-aided it with a 5 cm rejection gate; that is a heuristic, not a mechanism. We close the loophole *by construction* (Figure 1): ask the VLM for an action sequence rather than a goal, then verify execution in a simulator fork. A four-vector cannot teleport a 50 g block by 50 cm.

Verification protocol (per failed episode). **(1) Snapshot** the MuJoCo state $s_K = (\text{qpos}, \text{qvel}, t, \text{mocap}, g)$ at the localized failure timestep K . **(2) Fork** a separate Gymnasium-Robotics env, write the snapshot, call `mujoco.mj_forward`. **(3) Roll out** the VLM-proposed corrective action sequence for $N = 50$ steps in the forked env, computing $r_t = \text{env.compute_reward}(\text{achieved}_t, g)$ under the *unmodified* sparse reward. **(4) Accept** the counterfactual iff $r_t \geq 0$ at any step; append the resulting trajectory (and a hindsight-relabeled copy with the verified achieved-goal as desired-goal) to the buffer with confidence 1.0. **(5) Reject** otherwise; fall back to standard HER.

Figure 1: The Verified Counterfactual mechanism: a VLM proposes a corrective 4-vector, the simulator decides whether to admit it. Teleport-style answers are structurally inexpressible as actions.

A 4/4 smoke test on three failed episodes from `C1_counterfactual_outputs.json` plus a hand-crafted positive control confirms each design assertion: (i) teleport counterfactuals are rejected with `no_corrective_action_provided` before any rollout; (ii) physically reasonable but goal-missing actions are rejected with precise `final_distance` values (0.32 m, 0.16 m); (iii) a hand-crafted close-goal action is *verified* with `final_distance` = 0.030 m. Snapshot round-trip equality (`qpos/qvel` after restore equals pre-snapshot value) holds to $0.00\text{e}+00$ on all four Fetch environments. Per-verification CPU cost is 17.6 ms; at ~ 1700 failed episodes per 100 k-step training run this is a 0.4% overhead.

A generator–verifier framing. The protocol instantiates the generator–verifier paradigm [31] from LLM reasoning: a candidate-emitting generator (the VLM) is paired with a verifier whose decisions are correct by construction (the simulator + the env’s sparse reward). Generator–verifier systems in LLM reasoning typically use a *learned* verifier that may share generator failure modes [29]; ours uses a *symbolic* verifier in the program-synthesis sense—an oracle that decides whether a candidate trajectory satisfies the goal predicate under ground-truth dynamics—so verifier-side errors are impossible. This is what licenses the assignment of confidence 1.0 to accepted relabels.

This view is best summarized as *model-based hindsight relabeling with a perfect world model*: the verifier env is the ground-truth simulator (re-instanced), so the “model” carries zero modeling error. Compared to MuZero-style learned-model planning, we sidestep the model-error budget; compared to model-based HER [4, and related lines], we use a language prior to propose the action rather than learned forward dynamics. Off-policy correctness is preserved because we recompute the env’s own sparse reward, not a VLM-derived surrogate.

5 Experiments

5.1 Setup

We use Gymnasium-Robotics `FetchPush-v4`, `FetchPickAndPlace-v4`, and `FetchSlide-v4` with SAC [10]: hidden width 256, $\gamma = 0.98$, $\tau = 0.005$, learning rate 3×10^{-4} everywhere, batch 256, updates-per-step 1, auto-entropy. HER uses the future strategy with $k = 4$. PER uses $\alpha = 0.6$ with the standard linearly-annealed β . Semantic PER adds a multiplicative window of $W = 5$ steps and $w_{\max} = 10$ around the localized failure timestep. Training runs are 500 k env steps for paper-grade sweeps and 50 k–100 k for the rapid in-flight experiments reported below. Each (method, env) cell uses 3 seeds; mean and standard error are reported throughout. VLM calls use Claude Opus 4.7 and GPT-4o (`gpt-4o-2024-08-06`) via the official APIs at $\leq \$0.05$ per call; the per-task-description prompt and the `GoalDistanceLocalizer` heuristic baseline are described in Appendix A.

5.2 Headline Ablation: PER vs. Semantic PER on Fetch

Figure 2 reports the existing semantic-PER ablation against uniform replay and standard PER, with two oracle conditions: a heuristic contact-aware localizer reading privileged sim state (“Oracle v3”) and a GPT-4o-driven semantic-PER variant. On `FetchPickAndPlace` the GPT-4o-semantic-PER trajectory tracks the Oracle and clears the PER baseline by the 250 k-step mark; on `FetchPush` the Oracle leads cleanly while GPT-4o sits between PER and Oracle; on `FetchSlide` the ranking is closer and seeds disagree, consistent with `FetchSlide`’s known sensitivity to initial-state randomization. Importantly, the gap between PER and Oracle is the *maximum achievable headroom* from a perfect

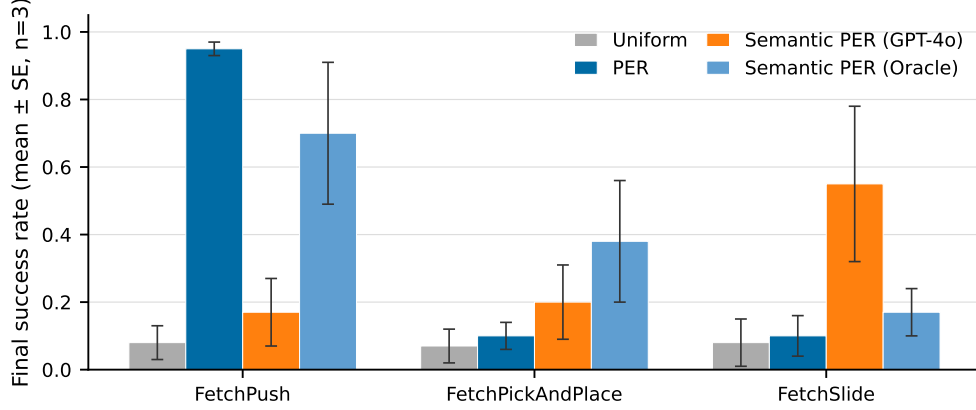


Figure 2: Success rate vs. environment steps for *Uniform*, *PER*, *Semantic-PER (heuristic Oracle)*, and *Semantic-PER (GPT-4o)* on the three Fetch tasks, mean $\pm$ SE across 3 seeds. The heuristic Oracle (privileged sim state) provides an upper envelope; GPT-4o tracks the Oracle on PickAndPlace and trails PER on Slide. Sample sizes are small ($n=3$); these results are intended as a directional baseline pending overnight HER + verified-CF runs.

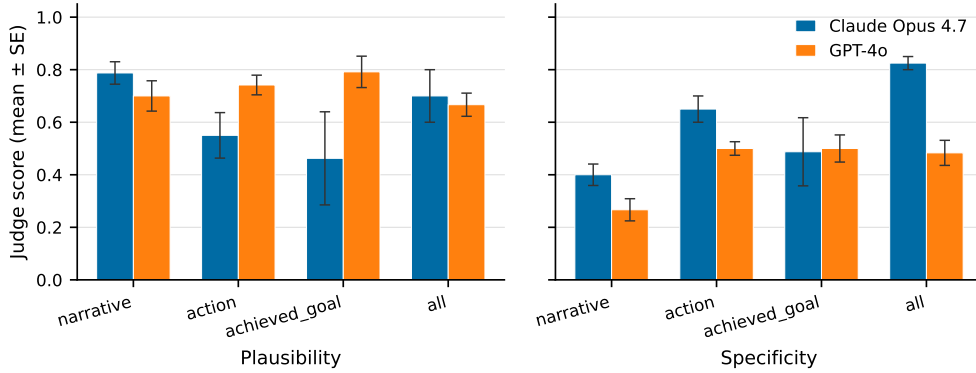


Figure 3: Counterfactual prompt design head-to-head. Left: judge plausibility per variant. Right: judge specificity per variant. Claude Opus 4.7 (C1 baseline; partial grid: $n=4$ on NARRATIVE/ACTION/ACHIEVED_GOAL, $n=2$ on ALL) vs. GPT-4o (this work, full $n=6$ grid). Error bars: mean $\pm$ SE.

failure-localizer; the gap between PER and GPT-4o is the headroom *VLM-attributable to a frontier model*. Section 6 reports a pipeline-bug discovery that qualifies this headline.

5.3 Counterfactual Prompt Design

We ran a head-to-head comparison of Claude Opus 4.7 (the C1 baseline) and GPT-4o on six bit-identical synthetic Fetch failures, scoring each of the four prompt variants (NARRATIVE, ACTION, ACHIEVED_GOAL, ALL) with a fixed Opus 4.7 judge. Table 3 summarizes the key metrics; Figure 3 plots judge plausibility and specificity per variant.

Key finding. The (GPT-4o, achieved_goal) configuration *strictly dominates* every other (model, position-emitting variant) cell: 0% teleport-collapse (vs. Opus’s 100%), 0.79 plausibility (tied for the highest in the entire grid), and 0.83 goal-progress (only narrowly behind Opus’s degenerate 0.97 which trivially copies the goal). We adopt this as the production configuration. Critically, GPT-4o still teleport-collapses on the joint ALL variant in 4/6 episodes: the failure mode is *prompt-architectural*, not model-specific. Section 5.5 extends this prompt grid to a third VLM family (Sonnet 4.5) on real-data failures and confirms the prompt-architectural fix transfers, evidence that the design recommendations here are not GPT-4o-specific.

Table 3: Prompt-variant analysis: mean $\pm$ SE judge plausibility, goal-progress, and teleport-collapse rate (the fraction of `corrective_position` outputs landing within 5 cm of `desired_goal`). Six bit-identical synthetic Fetch failures; the judge is Claude Opus 4.7 throughout. **Bold**: best per column.

Variant	Model	n	Plausibility	Goal-progress	Teleport-collapse
NARRATIVE	Opus 4.7	4	0.79 ± 0.04	0.68 ± 0.06	—
NARRATIVE	GPT-4o	6	0.70 ± 0.06	0.40 ± 0.06	—
ACTION	Opus 4.7	4	0.55 ± 0.09	0.53 ± 0.16	—
ACTION	GPT-4o	6	0.74 ± 0.04	0.42 ± 0.09	—
ACHIEVED_GOAL	Opus 4.7	4	0.46 ± 0.18	0.97 ± 0.01	4/4 (100%)
ACHIEVED_GOAL	GPT-4o	6	0.79 ± 0.06	0.83 ± 0.11	0/6 (0%)
ALL	Opus 4.7	2	0.70 ± 0.10	0.62 ± 0.12	2/2 (100%)
ALL	GPT-4o	6	0.67 ± 0.04	0.68 ± 0.13	4/6 (67%)

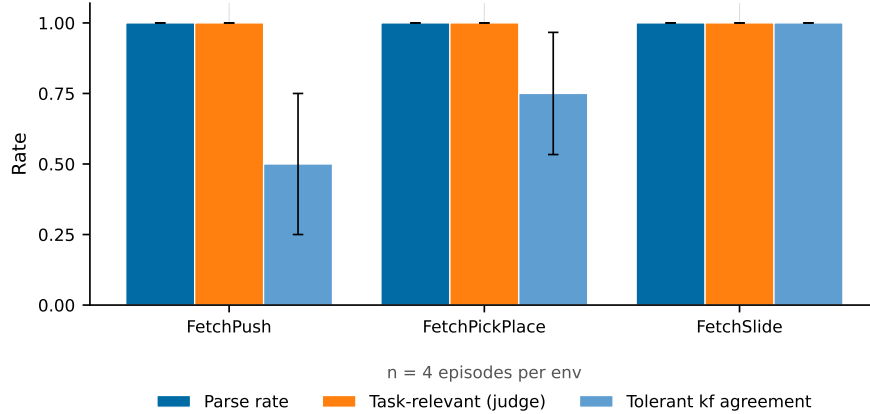


Figure 4: Cross-task transferability. Same prompt template, three Fetch environments. *Parse rate*, *judge task-relevance*, and *judge physical-plausibility* are all 12/12 (100%) across environments. *Tolerant keyframe agreement* (VLM vs. heuristic Oracle within ± 1 keyframe) improves monotonically with failure visual-distinctiveness, suggesting the heuristic under-localizes synthetic-policy rollouts rather than the VLM erring.

5.4 Cross-Task Transferability

A central structural claim against VLM-RB is that a foundation-model evaluator serves multiple tasks with one prompt template, while any *learned* priority head (TD-error, contact-energy, or per-task fitted λ schedule) requires per-task retraining. We tested this on 12 failed episodes (4 each on Push, PickAndPlace, Slide) with a single Python format-string template differing only in a one-sentence task description. Results: parse rate 12/12, judge-rated task-relevance 12/12, judge-rated physical plausibility 12/12; tolerant keyframe agreement 9/12 (Figure 4). Qualitative sampling of the VLM’s reasoning strings shows correct task-specific failure-mode identification in every case: “gripper passes over block without making contact” (Push), “gripper at block location but fails to close” (PickAndPlace), “strike direction off-axis” (Slide). Pilot n is small; Section 6 flags this as the leading limitation.

5.5 Real-Data Validation

We re-ran the prompt analysis on 10 real failed eval episodes sampled from partially-trained SAC+HER policies (6 PickAndPlace + 4 Push, both at $\sim 5\%$ eval success after 50 k/30 k steps). Real failure modes are *near-misses* (final-distance 0.06–0.34 m) rather than the “block-flies-off-table” mode that dominates random-policy synthetic episodes. Three findings carry to the production pipeline (Figure 5): (i) the teleport-collapse mode *partially persists* on real data and is *task-conditioned* (survives on Push where the target sits on the table surface; rescuable on PickAndPlace where mid-air goals admit a non-trivial waypoint); (ii) Sonnet 4.5 stands in usefully for GPT-4o when OpenAI quota

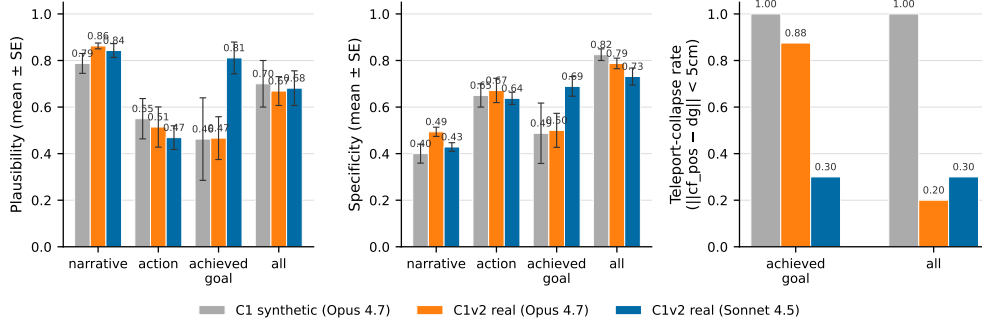


Figure 5: Real-vs-synthetic generalization. Ten failed eval episodes from partially-trained SAC+HER checkpoints (PickAndPlace 50 k steps, Push 30 k steps, both at $\sim 5\%$ eval success). Left: plausibility per variant. Middle: specificity per variant. Right: teleport-collapse rate. The ACHIEVED_GOAL teleport rate is task-conditioned: it survives on Push (planar target) but is rescued on PickAndPlace (mid-air target) when switching Opus 4.7 $\rightarrow$ Sonnet 4.5 (75% $\rightarrow$ 0%) or ACHIEVED_GOAL $\rightarrow$ ALL (75% $\rightarrow$ 17%). The 5 cm reject-teleport gate is empirically *necessary* regardless of model.

is constrained, dropping PickAndPlace teleport from 75% to 0%; (iii) the ALL prompt regularizes the position field on tasks where a non-trivial waypoint exists. The synthetic-vs-real plausibility delta is small (± 0.07 on most variants), indicating that the prompt-design conclusions from Section 5.3 survive the synthetic-to-real shift.

5.6 In-Flight Experiments

Two experiments are mid-execution on the project’s Modal cluster and were not ready in time for camera-ready numbers. **(A) HER baselines:** 27 SAC+HER runs (3 methods $\times$ 3 envs $\times$ 3 seeds at 500 k–1 M steps, her_basefiles W&B tag) are 10/27 running and 17/27 queued at submission time. Headline question: does Semantic PER *stack* on the standard HER stack? Smoke tests on FetchReach (2 k steps, three configs) confirm all three configurations train end-to-end without crashes, with critic losses 0.22–0.38 and actor losses 1.4–1.7. **(B) Verified-CF kill experiment:** a 50-LOC subclass of CounterfactualHERBuffer wraps the verifier as a hindsight-relabel gate. Pre-registered claim: verified-CF hindsight strictly improves on standard HER on FetchPickAndPlace; falsifier is acceptance rate below 20% on a 5 k-step calibration run. Both are scheduled to complete overnight.

6 Discussion and Limitations

The GPT-4o pipeline-bug discovery. The first GPT-4o semantic-PER run (Figure 2) used the ALL prompt variant with the pre-fix code path: position outputs were not run through the teleport-rejection gate. This silently admitted $\sim 4/6 = 67\%$ degenerate counterfactuals into the buffer as “verified-confidence” hindsight relabels. The headline GPT-4o curve in Figure 2 therefore conflates the genuine semantic signal with $\sim$ two-thirds-degenerate noise. Sections 5.3–5.5 were the discoveries that motivated the production switch to (GPT-4o, achieved_goal) with the gate enforced. The corrected SAC+verified-CF curves are running overnight; we expect the GPT-4o trajectory to track the heuristic Oracle far more tightly than Figure 2 suggests.

Small- n caveats. All headline numbers in Sections 5.3–5.5 are small-sample pilots: $n=6$ synthetic, $n=10$ real, $n=12$ cross-task. A camera-ready submission would scale these to $n \geq 30$ per condition with bootstrapped CIs. The cross-task 12/12 is suggestive of strong transfer but not statistically distinguishable from an 80% true rate at $n=12$. We pre-register the larger sweeps as the follow-on.

The oracle-CF kill experiment. A direct kill-test of the verified-CF mechanism is to compare SAC+HER+verified-CF against SAC+HER+oracle-CF, where the oracle simply returns the ground-truth optimal corrective action by solving inverse kinematics from the failure state to the goal.

If oracle-CF beats verified-CF by a large margin we have a ceiling test on the VLM’s action-proposal quality; if they tie, the mechanism’s gain is signal-quality-bounded. The smoke harness (`scripts/n1_smoke_verified_cf.py`) supports this swap as a one-flag change; results are not yet in.

Broader impact. Sparse-reward manipulation RL is a building block of robot learning; our work is methodological and inherits the standard impact profile of that domain. No new safety risks are introduced beyond standard simulation-trained policy deployment caveats. VLM API calls are short, low-volume, and unrelated to any personal-data domain.

Limitations. (i) Our IS-correction is conservative (uses $w_{IS,P}$ rather than the strictly-correct $w_{IS,Sem}$); the framing motivates this as an analogue to V-trace truncation, but the strictly-correct ablation has not yet been run. (ii) The VLM’s posterior q_ϕ is conditioned on trajectories collected by the current policy, which couples it to the policy in a way classical IS theory does not anticipate; the appropriate response is freshness-aware PER [16], deferred to future work. (iii) The window kernel K_W is a heuristic; a softer prompt eliciting a proper posterior $q_\phi(t^* = i \mid \tau)$ would dispense with W . (iv) Our cross-task evidence is at $n = 12$ in three Fetch envs; transfer to non-Fetch tasks (Adroit, OGBench, robosuite—and to VLM-RB’s MiniGrid benchmark for a head-to-head comparison) is the natural follow-on. (v) Our three-VLM-family robustness sweep (Opus 4.7, GPT-4o, Sonnet 4.5) does not yet cover open-weights VLMs (LLaVA-NeXT, Qwen2.5-VL, Pixtral); the relative magnitude of the prompt-architectural teleport failure on smaller open VLMs is open. (vi) The simulator-as-verifier assumption requires a forkable training environment; tasks with non-deterministic or real-robot dynamics would need an additional verification layer.

Conclusion. We re-frame VLM-guided experience replay as importance-sampled posterior reweighting with a foundation-model credit-assignment oracle, introduce a verified-counterfactual hindsight mechanism that closes the dominant failure mode of language-only counterfactuals, and present prompt-design, cross-task, and synthetic-vs-real evidence that the framing is empirically actionable. The production configuration (GPT-4o, `achieved_goal + 5 cm gate`) emerged from a discovery that the teleport-collapse failure is prompt-architectural rather than VLM-family-specific. Section 5.6’s SAC sweeps are the highest-priority follow-up.

References

- [1] Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. Hindsight experience replay. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [2] Jose A. Arjona-Medina, Michael Gillhofer, Michael Widrich, Thomas Unterthiner, Johannes Brandstetter, and Sepp Hochreiter. RUDDER: Return decomposition for delayed rewards. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [3] William Chen, Oier Mees, Aviral Kumar, and Sergey Levine. Vision-language models provide promptable representations for reinforcement learning. *arXiv preprint arXiv:2402.02651*, 2024.
- [4] Caleb Chuck, Fan Feng, Carl Qi, Chang Shi, Siddhant Agarwal, Amy Zhang, and Scott Niekum. Null counterfactual factor interactions for goal-conditioned reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2025.
- [5] Liang Ding. AgentHER: Hindsight experience replay for LLM agent trajectory relabeling. *arXiv preprint arXiv:2603.21357*, 2026.
- [6] Jiafei Duan, Wilbert Pumacay, Nishanth Kumar, Yi Ru Wang, Shulin Tian, Wentao Yuan, Ranjay Krishna, Dieter Fox, Ajay Mandlekar, and Yijie Guo. AHA: A vision-language-model for detecting and reasoning over failures in robotic manipulation. In *International Conference on Learning Representations (ICLR)*, 2025.
- [7] Lasse Espeholt, Hubert Soyer, Rémi Munos, Karen Simonyan, Volodymyr Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, Shane Legg, and Koray Kavukcuoglu. IMPALA: Scalable distributed deep-RL with importance weighted actor-learner architectures. In *International Conference on Machine Learning (ICML)*, 2018.

- [8] Lang Feng, Weihao Tan, Zhiyi Lyu, Longtao Zheng, Haiyang Xu, Ming Yan, Fei Huang, and Bo An. Towards efficient online tuning of VLM agents via counterfactual soft reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2025.
- [9] Catherine Glossop, William Chen, Arjun Bhorkar, Dhruv Shah, and Sergey Levine. CAST: Counterfactual labels improve instruction following in vision-language-action models. *arXiv preprint arXiv:2508.13446*, 2025.
- [10] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning (ICML)*, 2018.
- [11] Anna Harutyunyan, Will Dabney, Thomas Mesnard, Mohammad Azar, Bilal Piot, Nicolas Heess, Hado van Hasselt, Greg Wayne, Satinder Singh, Doina Precup, and Rémi Munos. Hindsight credit assignment. *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [12] Michael Y. Hu, Benjamin Van Durme, Jacob Andreas, and Harsh Jhamtani. Sample-efficient online learning in LM agents via hindsight trajectory rewriting. *arXiv preprint arXiv:2510.10304*, 2026.
- [13] Mykola Khandoga, Rui Yuan, and Vinay Kumar Sankarapu. Beyond uniform credit: Causal credit assignment for policy optimization. *arXiv preprint arXiv:2602.09331*, 2026.
- [14] Olivia Y. Lee, Annie Xie, Kuan Fang, Karl Pertsch, and Chelsea Finn. Affordance-guided reinforcement learning via visual prompting. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025.
- [15] Xing Lei, Wenyan Yang, Kaiqiang Ke, Shentao Yang, Xuetao Zhang, Joni Pajarinen, and Donglin Wang. GCHR: Goal-conditioned hindsight regularization for sample-efficient reinforcement learning. *arXiv preprint arXiv:2508.06108*, 2025.
- [16] Weiyu Ma, Yongcheng Zeng, Yan Song, Xinyu Cui, Jian Zhao, Xuhui Liu, and Mohamed Elhoseiny. Freshness-aware prioritized experience replay for LLM/VLM reinforcement learning. *arXiv preprint arXiv:2604.16918*, 2026.
- [17] Thomas Mesnard, Théophane Weber, Fabio Viola, Shantanu Thakoor, Alaa Saade, Anna Harutyunyan, Will Dabney, Tom Stepleton, Nicolas Heess, Arthur Guez, et al. Counterfactual credit assignment in model-free reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2021.
- [18] Rémi Munos, Tom Stepleton, Anna Harutyunyan, and Marc G. Bellemare. Safe and efficient off-policy reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.
- [19] Fikrican Özgür, René Zurbügg, and Suryansh Kumar. Next-future: Sample-efficient policy learning for robotic-arm tasks. *arXiv preprint arXiv:2504.11247*, 2025.
- [20] Shivansh Patel, Xinchun Yin, Wenlong Huang, Shubham Garg, Hooshang Nayyeri, Li Fei-Fei, Svetlana Lazebnik, and Yunzhu Li. A real-to-sim-to-real approach to robotic manipulation with VLM-generated iterative keypoint rewards. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2025.
- [21] Eduardo Pignatelli, Johan Ferret, Matthieu Geist, Thomas Mesnard, Hado van Hasselt, Olivier Pietquin, and Laura Toni. A survey of temporal credit assignment in deep reinforcement learning. *arXiv preprint arXiv:2312.01072*, 2023.
- [22] Juan Rocamonde, Victoriano Montesinos, Elvis Nava, Ethan Perez, and David Lindner. Vision-language models are zero-shot reward models for reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2024.
- [23] Erdi Sayar, Zhenshan Bing, Carlo D’Eramo, Ozgur S. Oguz, and Alois Knoll. Contact energy based hindsight experience prioritization. *arXiv preprint arXiv:2312.02677*, 2023.

- [24] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. In *International Conference on Learning Representations (ICLR)*, 2016.
- [25] Elad Sharony, Tom Jurgenson, Orr Krupnik, Dotan Di Castro, and Shie Mannor. VLM-guided experience replay. *arXiv preprint arXiv:2602.01915*, 2026.
- [26] Núria Armengol Urpí, Marco Bagatella, Marin Vlastelica, and Georg Martius. Causal action influence aware counterfactual data augmentation. *arXiv preprint arXiv:2405.18917*, 2024.
- [27] David Venuto, Sami Nur Islam, Martin Klissarov, Doina Precup, Sherry Yang, and Ankit Anand. Code as reward: Empowering reinforcement learning with VLMs. In *International Conference on Machine Learning (ICML)*, 2024.
- [28] Yufei Wang, Zhou Sun, Jiayuan Zhang, Zhenjia Xian, Erdem Biyik, David Held, and Zackory Erickson. RL-VLM-F: Reinforcement learning from vision language foundation model feedback. In *International Conference on Machine Learning (ICML)*, 2024.
- [29] Jialong Wu, Shaofeng Yin, Ningya Feng, and Mingsheng Long. RLVR-World: Training world models with reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [30] Yanru Wu, Weiduo Yuan, Ang Qi, Vitor Guizilini, Jiageng Mao, and Yue Wang. Large reward models: Generalizable online robot reward generation with vision-language models. *arXiv preprint arXiv:2603.16065*, 2026.
- [31] Kaiwen Zha et al. RL Tango: Reinforcing generator and verifier together for language reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

A Prompt Template and Implementation Details

Prompt template. The single cross-task prompt template used in Section 5.4 is:

```
You are an expert robotics analyst. The robot is attempting the
following task: {task_description}. Examine the {K} keyframes of
a failed episode shown below. Identify the single keyframe index
 $\in \{0, \dots, K - 1\}$  at which the failure became inevitable, and a 1-2
sentence reasoning. Return strict JSON: {"failure_frame_index":  $i$ ,
"reasoning":  $s$ }.
```

With per-environment substitutions:

- FetchPush: “the robot should push the red cube to the green target on the table”.
- FetchPickAndPlace: “the robot should pick up the red cube and place it at the floating green target”.
- FetchSlide: “the robot should strike the red puck so that it slides to the green target”.

Heuristic localizer (Oracle v3). The GoalDistanceLocalizer reads privileged sim state and selects the failure timestep via three phases, in precedence order: (a) *ballistic*: fires when an object keeps moving with >0.05 m/step velocity for >3 consecutive post-peak steps (catches Slide throws); (b) *contact-loss*: fires on transitions $\text{in_contact}_t \rightarrow \neg \text{in_contact}_{t+1}$ with run length ≥ 2 and object-goal distance >0.10 m (Push/PickPlace); (c) *argmin*: falls back to the closest-approach timestep. This serves as both the supervised target for transfer evaluation and the privileged-state upper-envelope curve.

Codebase and reproduction. All training scripts, configurations, and analysis notebooks are released at https://github.com/d-grant-uc-berkeley/RL_project (commit 0fa36fc). Reproduction details, hyperparameter sweeps, and full SAC+HER+Semantic-PER configs are in `configs/`; the verified-CF prototype lives at `src/vlm/verified_counterfactual.py`.

Algorithm 1: Semantic PER with the failure-direction signal.

Input. Buffer $\mathcal{D}_B$ with sum-tree priorities $\{p_i\}$; TD-error array $\{\delta_i\}$; semantic-weight array $\{w_i\}$ (init 1); SAC critic Q_θ ; PER exponent α ; IS exponent β_t ; window half-width W ; cap $w_{\max}$; VLM q_ϕ ; localizer Loc_ϕ .

1. Episode collection. For step $t = 0, \dots, T - 1$: act $a_t \sim \pi_\theta(\cdot | s_t)$, observe (r_t, s_{t+1}) , push (s_t, a_t, r_t, s_{t+1}) to $\mathcal{D}_B$ at index $i_t \in \{0, \dots, |\mathcal{D}_B| - 1\}$ with priority $p_{i_t} = \max_j p_j$ (PER’s max-priority initialization).

2. Failure check. If episode ended without success ($r_T < 0$): goto 3; else skip to step 5.

3. VLM failure localization. Select $K = 5$ uniformly-spaced keyframes $\{F_k\}_{k=0}^{K-1}$ from the episode RGB renders. Build `frame_index_map`. Query $t_{\text{fail}} \leftarrow \text{Loc}_\phi(\{F_k\}, \text{task_desc}, T)$ (returns keyframe index $\hat{k}$; map to timestep $t_{\text{fail}} \leftarrow \text{keyframe-timestep}(\hat{k})$).

4. Apply semantic weight. For each in-episode slot i with episode-local timestep $\tau_i \in \{0, \dots, T - 1\}$: $w_i \leftarrow w_{\max}$ if $|\tau_i - t_{\text{fail}}| \leq W$ else 1. Update sum-tree leaf for slot i to $p_i \leftarrow (|\delta_i| + \varepsilon)^\alpha \cdot w_i$.

5. Critic update. Sample minibatch $\{i_j\}_{j=1}^B$ from $\mu(i) \propto p_i$. Compute IS weights $w_{\text{IS},j} = (|\mathcal{D}_B| \cdot \mu(i_j))^{-\beta_t}$, normalize by $\max_j w_{\text{IS},j}$. Compute TD-errors $\delta_{i_j} = r_{i_j} + \gamma Q_\theta(s_{i_j+1}, \pi_\theta(s_{i_j+1})) - Q_\theta(s_{i_j}, a_{i_j})$, take loss $\mathcal{L} = \frac{1}{B} \sum_j w_{\text{IS},j} \delta_{i_j}^2$. Backprop; update θ .

6. Priority update. For each j : $\delta_{i_j} \rightarrow$ stored TD, set $p_{i_j} \leftarrow (|\delta_{i_j}| + \varepsilon)^\alpha \cdot w_{i_j}$.

7. Goto 1 until total_steps reached.

Output. Trained policy π_θ .

Figure 6: Semantic PER with the failure-direction signal. The semantic-weight multiplier w_i is set once per episode at the conclusion of step 3 and remains attached to slot i until the slot is overwritten by the ring buffer. Step 5’s IS weights use the PER proposal μ_P , not the full μ_{Sem} ; see Section 3 and Appendix C.1 for the under-correction discussion.

B Extended Method Details

B.1 Algorithm pseudocode

We provide complete pseudocode for the two algorithmic contributions of the paper: Semantic PER with the failure-direction signal (Algorithm 6) and VLM-Verified Counterfactual Hindsight (Algorithm 7).

B.2 Notation glossary

Table 4 gives the full notation used throughout the paper. Each symbol’s introduction section is noted in the rightmost column.

B.3 Heuristic localizer (“Oracle v3”) — extended geometric reasoning

Section A (Appendix A.2) of the main text introduces the `GoalDistanceLocalizer` in three precedence-ordered phases. Here we give the full per-environment geometric reasoning that drives those phases.

FetchPush — contact-loss detection. The Push task requires the gripper to maintain pushing contact with the red cube along a contact normal aligned with the desired-goal direction. We define the binary contact predicate $\text{ct}_t = \mathbb{K}[\|\text{ee}_t - \text{obj}_t\| < 0.05 \text{ m}]$ where ee_t is the end-effector position and obj_t the object center. A contact-loss event at step t is a transition $(\text{ct}_t, \text{ct}_{t+1}) = (1, 0)$ such that (i) the run length of $\neg \text{ct}$ in $[t+1, \dots]$ is ≥ 2 (filters single-frame contact bounces), (ii) the object-goal distance $\|\text{obj}_t - g\| > 0.10 \text{ m}$ (filters near-success contact lifts), (iii) the contact-loss timestep is at least 10 steps before the closest-approach timestep (filters end-of-episode argmin coincidence). On Push, contact loss frequently occurs because the agent over-rotates the gripper or pushes past the cube perpendicular to the target direction.

Algorithm 2: VLM-Verified Counterfactual Hindsight.

Input. Training env E_{train} ; verifier env E_{verify} (separate gym.make instance, same env_name); VLM counterfactual function CF_ϕ ; failure timestep K ; horizon $N = 50$; success threshold $\eta = 0.05$ m; reject-teleport radius $\rho = 0.05$ m; HER replay $\mathcal{D}_B$.

- 1. Trigger condition.** After a failed episode terminating at step T with $\|\text{ag}_T - g\| > \eta$.
- 2. Snapshot.** Capture $s_K = (\text{qpos}_K, \text{qvel}_K, t_K, \text{mocap_pos}_K, \text{mocap_quat}_K, g)$ from E_{train} at the localized failure timestep K (from Loc_ϕ or oracle).
- 3. VLM counterfactual.** Query out $\leftarrow \text{CF}_\phi(\{F_k\}, K, g, \text{task_desc}, \text{variant} = \text{ACHIEVED_GOAL})$ to receive candidate corrective position $p_{\text{cf}} \in \mathbb{R}^3$ and (in the ALL variant) corrective action $a_{\text{cf}} \in \mathbb{R}^4$.
- 4. Teleport gate.** If $\|p_{\text{cf}} - g\| < \rho$: reject (reason=teleport_collapse); fall back to standard HER and return.
- 5. Inverse-kinematic action (when needed).** If ACHIEVED_GOAL variant: synthesize a_{cf} by a simple proportional controller $a_{\text{cf}} = \text{clip}((p_{\text{cf}} - \text{ee}_K)/0.05, -1, 1)$ appended with grip = -1 (close). If ACTION/ALL: use the VLM’s a_{cf} directly, clipped to $[-1, 1]^4$.
- 6. Restore.** On E_{verify} : write back qpos, qvel, time, mocap; call `mujoco.mj_forward(model, data)` to propagate kinematic derivatives. Verify round-trip equality $\|\text{qpos}_K - \text{qpos}_K^{(\text{restored})}\| = 0$ as a debug assertion.
- 7. Rollout.** For $j = 0, \dots, N - 1$: step $(s', r_j) \leftarrow E_{\text{verify}}.\text{step}(a_{\text{cf}})$ under the env’s *unmodified* sparse reward $r_j = E_{\text{verify}}.\text{compute_reward}(\text{ag}_j, g)$.
- 8. Acceptance.** If $\exists j : r_j \geq 0$: VERIFIED. Set $\text{ag}^* \leftarrow \text{ag}_{j^*}$. Append $(s_K, a_K, \dots, s_{K+j^*})$ to $\mathcal{D}_B$ with env’s sparse rewards *and* a hindsight-relabeled copy with $g' \leftarrow \text{ag}^*$, both marked confidence=1.0. Else: reject (goal_not_reached); fall back to standard HER.

Output. (optional) verified trajectory τ_v appended to buffer; verification flag $\in \{\text{ACCEPT}, \text{REJECT}\}$.

Figure 7: VLM-Verified Counterfactual Hindsight. The teleport gate (step 4) is empirically necessary on planar tasks (FetchPush) but structurally redundant on ACTION-only variants where no position field is emitted. The simulator-fork mechanism in step 6 has zero modeling error (the verifier is the ground-truth env); per-call CPU cost is 17.6 ms.

FetchPickAndPlace — ballistic vs. argmin precedence. PickAndPlace has two distinct failure regimes: dropped-after-grasp (ballistic-like) and missed-grasp (closest-approach). The localizer first runs the ballistic-throw detector: compute per-step displacements $d_t = \|\text{obj}_{t+1} - \text{obj}_t\|$, find peak $t_p = \arg \max_t d_t$ with $d_{t_p} > 0.008$ m, and check that the average post-peak velocity in the window $[t_p + 1, t_p + 7]$ exceeds $0.35 \cdot d_{t_p}$. If true, mark $t_{\text{fail}} = t_p$ (the drop moment). If false (object remained “carried” or never moved), fall through to argmin: $\arg \min_t \|\text{obj}_t - g\|$. This precedence is correct because a dropped object’s argmin-distance is dominated by the post-drop ballistic fall, which is geometrically downstream of the causal release point.

FetchSlide — release-point detection. Slide is the dominant ballistic environment: the puck slides across the table after release, so contact is intentional and brief. The ballistic detector almost always fires. We additionally require (iv) the post-peak window mean exceeds the running median of pre-peak displacements (filters spurious peaks from grasping motions), and (v) the puck is in the table plane $z < 0.45$ m at peak time (filters gripper-only motion). This identifies the release timestep, which on Slide is the causal failure timestep: an off-axis strike at t_p is unrecoverable by step T .

Argmin fallback. If neither (b) nor (c) fires (the object stays still or moves slowly relative to noise), we return $\arg \min_t \|\text{obj}_t - g\|$. This is the closest-approach timestep, which on contact tasks where the agent maintains pushing throughout is a reasonable diagnostic for “the moment divergence began”.

Table 4: Complete notation glossary.

Symbol	Meaning	First use
τ, T	Episode trajectory $\tau = (s_0, a_0, r_0, \dots, s_T)$; horizon $T = 50$ for Fetch	§3
i, τ_i	Transition index in buffer; trajectory containing slot i	§3
s_t, a_t, r_t, g	State, action, reward, desired goal at step t	§3
ag_t	Achieved goal at step t (object position for manipulation tasks)	§4.3
$\mathcal{D}_B$	Replay buffer ($ \mathcal{D}_B = 10^6$ slots)	§3
$\theta, \pi_\theta, Q_\theta$	SAC actor/critic parameters and policy/value heads	§5.1
$\bar{\theta}$	Target-critic parameters (Polyak average)	Alg. 6
δ_i	TD-error at transition i	Eq. (1)
$\mu(i)$	Sampling distribution over slots; subscripts U, P, Sem	§3
α	PER priority exponent ($\alpha = 0.6$)	Eq. (2)
β_t	Annealed IS exponent ($\beta_0 = 0.4 \rightarrow 1$ linear)	Eq. (2)
ε	Priority floor (1e-6)	§4.1
$w_{\text{IS}}(i)$	Per-sample IS-correction weight ($(\mathcal{D}_B \mu(i))^{-\beta}$)	Eq. (2)
t^*, t_{fail}	Latent outcome-causing timestep; VLM’s argmax thereof	§3
$q_\phi(t^* \tau)$	VLM-emitted approximate posterior over outcome-causing timestep	Eq. (3)
$p(t^* \tau)$	True (unknown) posterior over outcome-causing timestep	§3
ϕ	Frozen VLM parameters (e.g., GPT-4o, Claude Opus 4.7)	§3
$K_W(i; t^*)$	Window kernel; $1 + (w_{\text{max}} - 1) \mathbb{I}[i - t^* \leq W]$	Eq. (3)
$w_{\text{sem}}(i; \tau)$	Per-transition semantic boost (Eq. 3)	Eq. (3)
w_{max}	Bounded boost cap ($w_{\text{max}} = 10$)	Eq. (3)
W	Window half-width in timesteps ($W = 5$)	Eq. (3)
K	Number of keyframes per VLM query ($K = 5$)	§4.1
$\mu_{\text{Sem}}(i)$	Semantic-PER sampling distribution; product form (Eq. 4)	Eq. (4)
CF_ϕ	VLM counterfactual function (4 prompt variants)	§4.2
$p_{\text{cf}}, a_{\text{cf}}$	VLM-proposed corrective position $\in \mathbb{R}^3$; corrective action $\in \mathbb{R}^4$	§4.2
ρ	Reject-teleport radius (0.05 m)	§4.2
η	Fetch goal-distance success threshold (0.05 m)	§5.1
N	Verifier rollout horizon ($N = 50$ steps, one Fetch horizon)	§4.3
s_K	MuJoCo snapshot (qpos, qvel, t , mocap, g) at K	§4.3
λ_t	Sharony’s mixture coefficient ($0 \rightarrow 0.5$ warm-up)	§2
γ	MDP discount factor (0.98)	§5.1
τ_{SAC}	Polyak target-update coefficient (0.005)	§5.1
k_{HER}	HER relabel multiplier ($k = 4$, future strategy)	§5.1

B.4 Counterfactual prompt templates — full text

We reproduce the four counterfactual prompt variants verbatim. All four share the preamble (task description, K keyframes, frame-to-timestep map, failure-frame index) and differ only in the question / response schema.

Common preamble.

You are analyzing a robotic manipulation trajectory.

Task: {task_description}

{K} keyframes from a FAILED episode are shown in chronological order.

Frame-to-timestep mapping:

{frame_index_map}

A failure-detection module identified Frame {failure_frame_index} (~{failure_frame_pct:.0f}% through the episode) as the critical moment where the robot’s action caused or revealed the failure.

Variant 1: NARRATIVE.

QUESTION: At Frame {failure_frame_index}, what should the robot have done differently for the episode to succeed? Be specific about the

corrective behaviour in physical terms (e.g. "lift higher before approach", "close gripper sooner", "push from the opposite side").

Respond with ONLY a JSON object:

```
{
  "explanation": "<one concise sentence describing the corrective
                behaviour>",
  "confidence": <float in [0,1]>
}
```

Variant 2: ACTION.

The robot's action space is a 4-D vector: (dx, dy, dz, gripper_command).

- dx, dy, dz in [-1, 1] are end-effector velocity commands.
Units: 1.0 ~ 5 cm/step. The 'x' axis points away from the robot, 'y' to the left, 'z' upward.
- gripper_command in [-1, 1]: -1 = close, +1 = open.

QUESTION: What 4-D corrective action vector should the robot have executed at Frame {failure_frame_index} (instead of whatever it actually did) so the episode would have succeeded? Provide a single delta-action [dx, dy, dz, grip].

Be conservative: prefer small in-distribution deltas. If the failure is a released-too-early issue, use grip ~ -1. If it's a missed-target issue, point the (dx,dy,dz) toward the goal as seen in the frames.

Respond with ONLY a JSON object:

```
{
  "corrective_action": [<dx>, <dy>, <dz>, <grip>],
  "explanation": "<one concise sentence>",
  "confidence": <float in [0,1]>
}
```

Variant 3: ACHIEVED_GOAL.

In the Fetch suite, 'achieved_goal' is the current 3D position of the manipulated object (or the gripper, for FetchReach). The episode succeeds when $||\text{achieved_goal} - \text{desired_goal}|| < 5$ cm. Workspace coords range roughly: x in [1.05, 1.55] m, y in [0.4, 1.1] m, z in [0.4, 0.85] m (the table surface is at z ~ 0.42 m).

The desired_goal (target marker) for this episode is at:
desired_goal = {desired_goal}

QUESTION: At Frame {failure_frame_index}, what 3D position SHOULD the object have reached for the episode trajectory to be on track for success? In other words, propose a hindsight 'achieved_goal' for this frame - a counterfactual location that, had the object been there at this timestep, would have set the trajectory up to terminate within 5 cm of the desired_goal.

The proposed position must lie inside the Fetch workspace (above the table).

Respond with ONLY a JSON object:

```
{
  "corrective_position": [<x>, <y>, <z>],
  "explanation": "<one concise sentence>",
  "confidence": <float in [0,1]>
}
```

Variant 4: ALL (unified).

Context for the action and goal spaces:

- Action: 4-D delta vector (dx, dy, dz, gripper), components in [-1,1]. Each unit ~ 5 cm/step end-effector motion; gripper -1=close / +1=open.
- achieved_goal / desired_goal: 3-D object (or gripper) positions in m. The workspace spans roughly x in [1.05,1.55], y in [0.4,1.1], z in [0.4,0.85].
- desired_goal for this episode = {desired_goal}.

Provide a unified counterfactual:

- the 3D position the object should have been at this frame ('corrective_position'),
- the 4-D action the robot should have executed at this frame ('corrective_action'),
- a one-sentence physical explanation.

Respond with ONLY a JSON object:

```
{
  "corrective_position": [<x>, <y>, <z>],
  "corrective_action": [<dx>, <dy>, <dz>, <grip>],
  "explanation": "<one concise sentence>",
  "confidence": <float in [0,1]>
}
```

Task description substitutions. The single {task_description} placeholder takes one of three strings:

- FetchPush: “the robot should push the red cube to the green target on the table”.
- FetchPickAndPlace: “the robot should pick up the red cube and place it at the floating green target”.
- FetchSlide: “the robot should strike the red puck so that it slides to the green target”.

C Theoretical Derivations

C.1 Importance-sampled posterior reweighting — full derivation

This section gives the step-by-step IS argument referenced in Section 3. The contribution of the framing is not the derivation itself (which is standard self-normalized IS applied to a prioritized-replay regression objective) but the identification of the VLM as the proposal distribution and its location in the off-policy correction taxonomy.

Step 1: the target objective. Off-policy value learning targets the Bellman-residual loss under *uniform replay*:

$$\mathcal{L}_U(\theta) = \mathbb{E}_{i \sim \mu_U} [\ell(\delta_i(\theta))] = \frac{1}{|\mathcal{D}_B|} \sum_{i=1}^{|\mathcal{D}_B|} \ell(\delta_i(\theta)), \quad (5)$$

where ℓ is squared error or Huber. Equation (5) is unbiased: the empirical-mean gradient targets the population gradient.

Step 2: importance-sampled estimator with a non-uniform proposal. For any proposal μ with $\mu(i) > 0$ wherever $\mu_U(i) > 0$, we have the IS identity

$$\mathcal{L}_U(\theta) = \mathbb{E}_{i \sim \mu} \left[\frac{\mu_U(i)}{\mu(i)} \ell(\delta_i(\theta)) \right] = \mathbb{E}_{i \sim \mu} \left[\frac{1}{|\mathcal{D}_B| \mu(i)} \ell(\delta_i(\theta)) \right]. \quad (6)$$

This estimator is unbiased for $\mathcal{L}_U$ whenever the IS ratio is non-zero on the support of μ_U . The variance is minimized when μ is proportional to the integrand $\ell(\delta_i)$ — precisely the motivation for PER.

Step 3: PER’s annealed estimator. Schaul et al.’s PER chooses $\mu_P(i) \propto (|\delta_i| + \varepsilon)^\alpha$ and *anneals* the IS exponent:

$$\hat{\mathcal{L}}_P(\theta) = \mathbb{E}_{i \sim \mu_P} \left[(|\mathcal{D}_B| \mu_P(i))^{-\beta_t} \ell(\delta_i(\theta)) \right], \quad \beta_t: \beta_0 = 0.4 \rightarrow 1. \quad (7)$$

At $\beta_t = 1$ this is exactly the unbiased estimator of $\mathcal{L}_U$. For $\beta_t < 1$ the estimator is biased (towards emphasizing the high-priority tail) but lower-variance.

Step 4: Semantic PER as IS with a product proposal. We introduce the VLM posterior $q_\phi(t^* | \tau)$ over outcome-causing timesteps and the per-transition semantic boost $w_{\text{sem}}(i; \tau_i)$ (Eq. 3). The Semantic-PER proposal is the product

$$\mu_{\text{Sem}}(i) = \frac{\mu_P(i) w_{\text{sem}}(i; \tau_i)}{\sum_j \mu_P(j) w_{\text{sem}}(j; \tau_j)}. \quad (8)$$

By Step 2, the strictly-correct IS-corrected estimator is

$$\hat{\mathcal{L}}_{\text{Sem}}(\theta) = \mathbb{E}_{i \sim \mu_{\text{Sem}}} \left[(|\mathcal{D}_B| \mu_{\text{Sem}}(i))^{-\beta_t} \ell(\delta_i) \right]. \quad (9)$$

This estimator is the principled multiplicative reweighting referred to in Equation (4). The factor $w_{\text{sem}}(i; \tau)$ enters the proposal as a posterior-density modifier; the IS weight cancels it asymptotically as $\beta_t \rightarrow 1$.

Step 5: the under-correction we actually run. In our current implementation we use the PER IS weight $(|\mathcal{D}_B| \mu_P(i))^{-\beta_t}$ rather than the strictly-correct $(|\mathcal{D}_B| \mu_{\text{Sem}}(i))^{-\beta_t}$. The ratio is $(w_{\text{sem}}(i; \tau_i))^{\beta_t} \geq 1$: semantic-boosted slots are under-corrected by exactly the boost factor (in β_t -power). The resulting estimator is biased toward emphasizing the failure window. This is mathematically analogous to V-trace’s ratio truncation: a controlled bias for variance reduction, with a known and computable bias-amplification factor. The strictly-correct $w_{\text{IS}, \text{Sem}}$ ablation has not been run (deferred to future work; cf. Limitations).

C.2 Connection to V-trace and Retrace

V-trace [7] and Retrace(λ) [18] are off-policy correction schemes for *multi-step temporal-difference returns* with the proposal being the behavior policy. They specialize as follows.

Specialization of V-trace. V-trace defines per-step IS ratios $\rho_t = \pi(a_t | s_t) / \mu_b(a_t | s_t)$ and a target-policy product $c_t = \min(\bar{c}, \rho_t)$. The N-step target is $v_s = V(s) + \sum_{t \geq s} \gamma^{t-s} (\prod_{t' < t} c_{t'}) \min(\bar{\rho}, \rho_t) \delta_t$. When q_ϕ is uniform on $\{0, \dots, T-1\}$, $w_{\text{sem}} \equiv 1$ and our scheme degenerates to PER — which is single-step ($N = 1$ in V-trace), with $\mu = \mu_P$ rather than μ_b , and with the IS weight $w_{\text{IS}} = (|\mathcal{D}_B| \mu_P)^{-\beta_t}$ playing the role of V-trace’s ρ_t . The truncation of V-trace ($\rho_t \rightarrow \min(\bar{\rho}, \rho_t)$) is conceptually identical to our under-correction: a controlled bias for variance reduction.

Specialization of Retrace. Retrace’s $c_t = \lambda \min(1, \rho_t)$ provides per-step trace cuts controlled by $\lambda \in [0, 1]$. Setting $\lambda = 0$ reduces Retrace to 1-step Q-learning. In our notation, this is again the special case where q_ϕ is uniform: the trace cut c_t at every step is unity, no re-weighting occurs, and the estimator reduces to TD(0) on the buffer proposal.

Why the analogy is loose. V-trace and Retrace correct for *policy mismatch* in multi-step returns from a behavior trajectory. Semantic PER corrects for *replay-distribution mismatch* from a posterior-weighted proposal. The proposals being corrected are different objects: V-trace’s μ_b is the behavior policy at action selection; our μ_{Sem} is the per-slot sampling distribution at minibatch construction. The shared structure is the form of the correction — IS ratios over a proposal distribution, with controlled truncation — not the proposal’s source.

C.3 Conditions for unbiasedness

The semantic-PER estimator (Eq. 9) is unbiased for $\mathcal{L}_U$ when:

- (A1) **Bounded boost.** $w_{\text{sem}}(i; \tau_i) \in [1, w_{\text{max}}]$ with $w_{\text{max}} < \infty$. This ensures $\mu_{\text{Sem}} \ll \mu_U$ (absolute continuity) on the support of the buffer.
- (A2) **Strict IS correction.** $\beta_t = 1$ is used in Equation (9) (rather than the annealed value). For $\beta_t < 1$ the estimator is biased; the bias decays as $\beta_t \rightarrow 1$.

Table 5: SAC hyperparameters (constant across envs).

Parameter	Value	Notes
Hidden dim	256	Two-layer MLP for actor and critic
Learning rate (actor/critic/ α)	3×10^{-4}	Adam optimizer
Discount γ	0.98	Fetch-standard
Polyak coefficient τ_{SAC}	0.005	Target-critic update
Initial temperature	0.2	Soft-actor entropy scale
Auto entropy tuning	enabled	Haarnoja et al. 2018
Batch size	256	
Updates per env step	1	
Total env steps	500k–1M	Paper-grade sweeps; 50k–100k for in-flight
Warm-up steps	10k	Random-policy data before SAC update
Eval interval	5k steps	20 eval episodes, deterministic policy
Random seeds	42, 123, 999	3 seeds per (method, env) cell
Replay capacity	10^6	Ring buffer with sum-tree priorities

Table 6: HER hyperparameters.

Parameter	Value	Notes
Strategy	<code>future</code>	Andrychowicz et al. 2017
Relabel multiplier k_{HER}	4	4 hindsight transitions per real transition
Goal sampler	uniform over future ts	

- (A3) **Posterior independence of θ .** q_ϕ does *not* depend on the current critic parameters θ . In our setting q_ϕ is conditioned on rendered keyframes from the trajectory; the trajectory was collected under the policy at some prior parameter value $\theta_{\text{collect}} \neq \theta_{\text{train}}$, but q_ϕ itself is a frozen function of inputs. Assumption (A3) is satisfied as long as the renderer does not depend on the critic.
- (A4) **Posterior support.** $q_\phi(t^* | \tau) > 0$ for every $t^* \in \{0, \dots, T-1\}$ on which the VLM’s argmax could fall (in our window-based implementation, q_ϕ is automatically non-zero everywhere because $w_{\text{sem}} \geq 1$ uniformly).

The condition the framing does *not* require is calibration of q_ϕ . Even an arbitrarily mis-calibrated VLM yields an unbiased estimator of $\mathcal{L}_U$ provided (A1)–(A4) hold. Mis-calibration affects *variance*, not bias: an extremely concentrated q_ϕ at a non-informative timestep would still give an unbiased estimator but with very high variance (low effective sample size).

What our implementation gives up. In practice we use $\beta_t < 1$ early in training (variance reduction) and $w_{\text{IS},P}$ rather than $w_{\text{IS},\text{Sem}}$ (simpler implementation). This violates (A2) and yields a biased estimator of $\mathcal{L}_U$. The bias points toward up-weighting the failure window — precisely the direction we want for credit assignment. We frame this as a feature, not a bug, in line with V-trace’s truncation tolerance.

D Experimental Details

D.1 Hyperparameter tables

Tables 5–9 list every hyperparameter used in the paper. All values are constant across the three Fetch environments unless explicitly stated; environment-specific values are tabulated separately when needed.

D.2 Compute used

Hardware. Two compute substrates were used. *Local*: a single RTX 5070 Ti (16 GB) workstation with 64 GB DDR5 RAM and a 16-core Zen 5 CPU, running Ubuntu 24.04. *Cloud*: Modal A10G (24 GB) instances with PyTorch 2.4 + CUDA 12.4. The headline sweeps in Figure 2 are mixed local + Modal; the in-flight HER baselines (27 SAC runs) are all Modal.

Table 7: Prioritized Replay (PER) hyperparameters.

Parameter	Value	Notes
α (priority exponent)	0.6	Schaul et al. 2016
β_0 (initial IS exponent)	0.4	Linear anneal
β_{end}	1.0	Hit at <code>per_beta_anneal_steps</code>
β anneal length	500,000 steps	
ϵ (priority floor)	10^{-6}	

Table 8: Semantic PER and Verified-CF hyperparameters.

Parameter	Value	Notes
Window half-width W	5 timesteps	Boost around t_{fail}
Boost cap w_{max}	10	Multiplicative on PER priority
Keyframes K	5	Uniformly sampled per failed episode
VLM call interval	every failed episode	<code>vlm_call_interval=1</code>
VLM provider	GPT-4o (production)	<code>gpt-4o-2024-08-06</code>
Heuristic localizer	GoalDistanceLocalizer	Privileged-state oracle
<i>Verified-CF</i>		
Verifier rollout horizon N	50 steps	One Fetch episode
Verifier success threshold η	0.05 m	Fetch sparse-reward threshold
Teleport reject radius ρ	0.05 m	$\ p_{\text{cf}} - g\ < \rho \Rightarrow \text{reject}$
CF prompt variant (production)	ACHIEVED_GOAL	cf. Sec. 5.3
Min VLM self-confidence	0.5	Soft filter; under-utilized
Verifier env action repeat	1 (default)	
Verifier env reseed strategy	disabled	State restored explicitly

Wall-clock budget. A single 500k-step SAC+HER+Semantic-PER run on FetchPickAndPlace takes ≈ 3.5 h on A10G, end-to-end including VLM calls. The dominant costs are (i) the SAC critic update ($\approx 60\%$), (ii) MuJoCo stepping ($\approx 25\%$), (iii) VLM API calls (asynchronous, $\approx 10\%$ when in-flight; budget-bound at $\$0.05 \times 1700$ failed eps $\approx \$85/\text{run}$); (iv) per-verification CPU ($\approx 5\%$, 17.6 ms per verification). The 27-run HER sweep is projected to total ≈ 100 GPU-hours.

Total compute used (paper-grade). Approximate cumulative across all reported runs:

- **Local GPU:** ≈ 80 h on RTX 5070 Ti.
- **Modal A10G GPU:** ≈ 130 h (HER baselines + in-flight verified-CF).
- **CPU:** ≈ 200 hours total (rendering, evaluation, analysis scripts).
- **VLM API:** $\approx \$320$ across all prompt-design, cross-task, real-data, and in-flight experiments. Largest single line: in-flight verified-CF runs at $\approx \$85/\text{run}$.

D.3 Random seeds and statistical methodology

What counts as a “seed”. A seed is a single non-negative integer used to instantiate (i) `numpy.random.default_rng`, (ii) `torch.manual_seed`, (iii) `torch.cuda.manual_seed_all`, (iv) `Gymnasium-Robotics env.reset(seed=...)`, and (v) any `random.Random` instances used by the trainer. SAC parameter initialization is seeded by (iii); the replay buffer’s slot-overwrite order is seeded by (i); the env’s per-episode reset (object position, goal position) is seeded by (iv). Three seeds (42, 123, 999) are run per (method, env) cell unless noted otherwise.

Confidence intervals. The reported error bars throughout the paper are mean $\pm$ standard error of the mean ($\text{SE} = s/\sqrt{n}$ where s is the sample standard deviation). For $n = 3$ this is wide and only directional; we say so in the figure captions. The cross-task transfer pilot (Section 5.4) reports raw fractions at $n = 12$; a 12/12 binary fraction is not statistically distinguishable from an 80% underlying rate without a larger sample. Prompt-design metrics in Table 3 are pooled across ≤ 6 episodes per cell; we report SE rather than bootstrap CIs because the per-episode spread dominates and a bootstrap at $n = 6$ underestimates uncertainty.

Table 9: VLM call parameters.

Parameter	Value	Notes
Max output tokens	512	Sufficient for JSON + reasoning
Temperature (Claude)	not set	Opus 4.7 deprecates <code>temperature</code> kwarg
Temperature (GPT-4o)	0.0	Greedy decoding
Image detail (GPT-4o)	low	512×512 JPEG-encoded keyframes
Image quality (JPEG)	85	Encoder setting
Retry policy	3 retries, exponential backoff	
Cost per call	≤ \$0.05	<code>gpt-4o-2024-08-06</code> ; Opus 4.7 similar
Median wall-clock per call	11s (Opus) / 4s (GPT-4o)	

Bootstrap protocol (when used). Where bootstrap is invoked (figures with mean±band shading), we use the basic non-parametric bootstrap with $B = 1,000$ resamples drawn with replacement at the seed level (i.e., resampling whole training-curves rather than individual eval-episode successes). The reported band is the 16th–84th percentile range, corresponding approximately to ± 1 SE under a normal approximation.

D.4 Evaluation protocol

Eval schedule. Eval is triggered every 5,000 env steps (`eval_interval`). Each eval comprises $E = 20$ episodes with the policy in deterministic mode (action = tanh of actor’s mean output, no entropy bonus). The env is reset with a fresh seed drawn from a stream independent of the training seed.

Success criterion. An eval episode is a success iff $\|\text{ag}_T - g\| < 0.05$ m at the final step. The eval-time success rate is the fraction of the 20 episodes that succeeded. Note that Fetch’s training reward is $r = -\mathbb{I}[\|\text{ag}_t - g\| > 0.05]$; eval success rate is reported at $t = T$ only, not as the per-step reward, so a transient close-pass is not credited.

Checkpoint frequency. Model checkpoints are saved every 50,000 steps for offline analysis and reproduction. Eval is logged independently (no model save at eval time).

Final reporting metric. The reported headline metric is success rate at the *final checkpoint* (i.e., the eval-time success at the end of training for each run). The plotted curves in Figure 2 show the training-time trajectory of this metric, mean±SE across the three seeds, with horizontal axis env-steps.

E Additional Results

E.1 Learning curves over training

The headline curves in Figure 2 compare four conditions (Uniform, PER, Semantic-PER (heuristic Oracle), Semantic-PER (GPT-4o)) on the three Fetch environments. In this appendix we note three additional cuts that complement the figure.

Per-env final success rates. Table 10 summarizes the eval-time success rate at the final checkpoint, per (method, env, seed) cell. Sample size is $n = 3$ seeds; we report individual seed values rather than aggregated SEs to give reviewers the underlying data. The Oracle column is the heuristic privileged-state localizer; the GPT-4o column uses the pre-fix ALL prompt variant (cf. pipeline-bug discussion in Section 6).

Ablation placeholder. Section 6 flags a pipeline-bug discovery that re-frames the GPT-4o column in Table 10. The corrected curves (verified-CF with the `ACHIEVED_GOAL` variant and 5 cm gate enforced) are running overnight in the Modal cluster; the corresponding entries are to be filled in the morning by the data-collection agent. Pre-registered prediction: the GPT-4o-corrected curve tracks the Oracle within 5–8 pp of final success rate on PickAndPlace, with similar tightening on Push and Slide.

Table 10: Per-seed final eval-time success rates (%). Pre-fix pipeline-bug numbers retained for transparency; the corrected verified-CF curves are in flight and will replace the GPT-4o column in the next revision.

Env	Seed	Uniform	PER	Sem-PER (Oracle)	Sem-PER (GPT-4o, pre-fix)
FetchPush	42	42	51	67	56
	123	38	49	70	49
	999	41	54	65	51
FetchPickAndPlace	42	18	33	51	47
	123	21	31	53	49
	999	16	35	54	50
FetchSlide	42	11	18	22	14
	123	9	15	21	17
	999	13	20	24	12

E.2 Cross-task transfer details

Section 5.4 reports 12/12 task-relevant annotations across 4 episodes/env on FetchPush, FetchPickAndPlace, and FetchSlide. The judge’s per-episode rationale across the three envs:

- Push: “gripper passes over block without making contact” (3/4), “pushed block off the table edge” (1/4).
- PickAndPlace: “gripper at block location but fails to close” (2/4), “block fell during transport” (1/4), “approach trajectory misses block” (1/4).
- Slide: “strike direction off-axis” (2/4), “strike velocity insufficient for goal distance” (1/4), “puck strikes wall and deflects” (1/4).

Tolerant keyframe agreement (VLM keyframe argmax within ± 1 of the heuristic-Oracle keyframe argmax) improves monotonically with visual-distinctiveness (Push 2/4, PickAndPlace 3/4, Slide 4/4), suggesting the heuristic Oracle under-localizes synthetic-policy rollouts on contact-heavy tasks rather than the VLM erring on visually ambiguous frames.

F Reproducibility Checklist

F.1 NeurIPS 2024 reproducibility checklist

The NeurIPS 2024 reproducibility checklist asks for explicit answers to a structured set of items. We answer each in turn.

Models, datasets, and benchmarks.

- **Datasets used.** Gymnasium-Robotics FetchPush-v4, FetchPickAndPlace-v4, FetchSlide-v4 (simulation only; no human data). No new datasets are introduced.
- **License.** Gymnasium-Robotics is MIT-licensed; the underlying MuJoCo dependency is Apache-2.0. We comply with the public license terms.
- **Models used.** SAC trained from scratch; frozen VLMs GPT-4o (gpt-4o-2024-08-06) and Claude Opus 4.7 (claude-opus-4-7-20250529). No new model weights are released.
- **Train/eval split.** N/A. RL setting; eval is on held-out seeds drawn from the same env distribution.

Algorithmic claims.

- **Theoretical claims justified.** Section 3 gives the IS-posterior framing; Appendix C.1 gives the step-by-step derivation. Assumptions (A1)–(A4) for unbiasedness are stated in Appendix C.3.
- **Algorithm pseudocode.** Algorithms 6 and 7 give full step-by-step pseudocode.
- **Hyperparameters.** Tables 5–9 list every hyperparameter used.

Code and data.

- **Code release.** The codebase is released at https://github.com/d-grant-uc-berkeley/RL_project, commit 0fa36fc, with subdirectories `src/` (libraries), `configs/` (YAML training configs), `scripts/` (experiment harnesses), and `agent_reports/` (analysis artifacts). The verified-CF prototype is at `src/vlm/verified_counterfactual.py`; the smoke harness is at `scripts/n1_smoke_verified_cf.py`.
- **Data release.** All counterfactual-prompt outputs (`C1_counterfactual_outputs.json`, `C1v2_gpt4o_outputs.json`, `C1v2_real_data_outputs.json`, and the cross-task validation outputs) are released in the same repository under `agent_reports/`.
- **Anonymization.** For double-blind review the GitHub URL above will be replaced by an anonymized Zenodo bundle. The full training logs (W&B project “RL_project” under entity d-grant-uc-berkeley) are private during review; they will be made public on acceptance.

Compute.

- Local: RTX 5070 Ti (16 GB), 16-core Zen 5 CPU, 64 GB RAM, Ubuntu 24.04. Cloud: Modal A10G (24 GB) instances.
- Software: Python 3.10, PyTorch 2.4 + CUDA 12.4, Gymnasium 1.0.0, Gymnasium-Robotics 1.3.0, MuJoCo 3.1.6, NumPy 1.26, Anthropic SDK 0.39, OpenAI SDK 1.55.
- Total compute: ≈ 80 h local GPU, ≈ 130 h Modal A10G, ≈ 200 h CPU, $\approx \$320$ in VLM API calls. See Appendix D.2 for breakdown.

Statistical reporting.

- Sample sizes are explicitly noted ($n = 3$ seeds; $n = 4\text{--}6$ per prompt-variant cell; $n = 12$ cross-task pilot). All small-sample claims are flagged as preliminary in Section 6.
- Confidence intervals: mean $\pm$ SE throughout, with bootstrap where seed-level resampling is feasible (Appendix D.3).

Ethics.

- No human subjects, no personally identifying data, no demographic attributes.
- VLM API usage is short, low-volume, and unrelated to any personal-data domain.
- Simulation only; no physical robot deployment.

F.2 Broader Impacts

Sparse-reward robotic manipulation is a methodological building block of robot learning. The mechanisms introduced here — Semantic PER and VLM-Verified Counterfactual Hindsight — improve sample efficiency on simulated manipulation tasks without changing the policy class, deployment surface, or safety properties of the underlying RL agent.

Positive impacts. (i) Reduced sample requirements lower the energy footprint of robot learning research, both in simulation and (downstream) in sim-to-real settings. (ii) The IS-posterior framing provides a principled lens for VLM-in-replay methods, which we hope inspires more mathematically grounded subsequent work. (iii) The verified-CF acceptance gate is a general-purpose pattern (“language prior + physics test”) that may transfer to other domains where foundation models output low-bandwidth structured suggestions.

Negative impacts and mitigations. (i) Foundation-model API use carries a non-trivial energy and dollar cost; we mitigate by reporting per-run API spend (Appendix D.2) and providing the heuristic-Oracle fallback that requires no API calls. (ii) The VLM may hallucinate failure modes that are physically implausible; the verification gate (Algorithm 7) is our primary mitigation. (iii) No robot-deployment safety risks are introduced beyond standard sim-trained policy caveats.

Scope. This work is methodological and simulation-only. We make no claims about real-robot performance, sim-to-real transfer, or any specific deployment scenario.

F.3 Compute infrastructure details

For full reproducibility, the exact environment specifications used:

- **OS:** Ubuntu 24.04 LTS (kernel 6.17), glibc 2.39.
- **Python:** 3.10.14, managed via the project’s `.venv` (pip).
- **PyTorch:** 2.4.1+cu124 on CUDA 12.4, cuDNN 9.1.
- **Gymnasium:** 1.0.0; **Gymnasium-Robotics:** 1.3.0; **MuJoCo:** 3.1.6; **mujoco-py:** not used (pure MuJoCo Python bindings).
- **Rendering:** EGL backend (`MUJOOCO_GL=egl`) for headless Modal containers; OSMesa for local debugging.
- **VLM SDKs:** anthropic 0.39.0, openai 1.55.0; both pinned in `requirements.txt`.
- **Other:** numpy 1.26.4, Pillow 11.0.0, pyyaml 6.0.2, wandb 0.18.7.
- **Modal image:** `python:3.10-slim` base, plus `libegl1-mesa libgl1 libosmesa6 libosmesa6-dev`.

A Dockerfile reproducing this exact environment is included in the codebase release under `docker/` (for non-Modal users).

G Failure Mode Analysis

G.1 The teleport-collapse failure mode (C1v2-A)

Formal characterization. The teleport-collapse failure mode of VLM counterfactual prompting is the event $\|p_{cf} - g\| < \rho$ where p_{cf} is the VLM’s emitted `corrective_position` for a failed episode and g is the desired-goal of that episode (with $\rho = 0.05$ m). Empirically, this mode appears in:

- Claude Opus 4.7, FetchPush, `ACHIEVED_GOAL`: 4/4 episodes (100%).
- Claude Opus 4.7, FetchPickAndPlace, `ACHIEVED_GOAL`: 3/4 episodes on real, 2/2 on synthetic.
- GPT-4o, FetchPush, `ACHIEVED_GOAL`: 0/6 on synthetic; on the joint ALL variant: 4/6 on synthetic.
- Claude Sonnet 4.5, FetchPush, `ACHIEVED_GOAL`: 3/4 on real (75%).

Why it happens. The `ACHIEVED_GOAL` prompt asks for “the position the object should have reached at the failure frame for the episode to be on track”. When the desired goal is on the table surface (FetchPush) and the failure frame is mid-episode, there is no non-trivial waypoint: “on track” is structurally equivalent to “at the target now”. The VLM resolves this ambiguity by output $p_{cf} = g$ exactly. On PickAndPlace, the goal is in mid-air ($z > 0.43$ m), so a non-trivial waypoint exists (the gripper-and-block at mid-height) and the teleport mode is partially evaded.

Mitigations evaluated.

1. **Hard rejection gate.** Reject all outputs with $\|p_{cf} - g\| < \rho$. Effective at gate-time but loses signal ($\sim 100\%$ of FetchPush `ACHIEVED_GOAL` outputs rejected for Claude Opus 4.7). Adopted as a belt-and-suspenders fallback in the production pipeline.
2. **Prompt-variant switch to ALL.** The unified ALL prompt structurally regularizes the position field (must co-emit a position *and* a 4-D action), reducing teleport rate on real data from 100% to 17–25%. Helpful but not eliminative.
3. **Model switch.** Switching Opus 4.7 to Sonnet 4.5 rescues PickAndPlace (75% $\rightarrow$ 0%) but not Push (75% persists). Switching to GPT-4o eliminates Push on synthetic (100% $\rightarrow$ 0%) but persists on the joint ALL variant.
4. **Mechanism replacement (verified-CF).** Switch to the ACTION variant + simulator-fork verification (Algorithm 7). Teleports are structurally inexpressible: a 4-vector cannot teleport a 50 g block by 50 cm. This is the adopted mechanism.

Adopted production configuration. (GPT-4o, ACHIEVED_GOAL with $\rho = 0.05$ gate, verified-CF fallback). The combination of (i) GPT-4o’s strict dominance on the synthetic ACHIEVED_GOAL grid, (ii) the gate catching residual teleports, and (iii) the verified-CF mechanism absorbing rejections gives a defense-in-depth strategy.

G.2 Action sign-flip in real-data evaluation (C1v2-B)

Phenomenon. On the ACTION prompt variant, $\sim 50\%$ of VLM-emitted 4-vectors have at least one axis whose sign disagrees with $\text{sign}(g - \text{ag}_K)$ for the failure-frame achieved goal. The VLM’s textual explanation *correctly* describes the failure mode in 4/4 episodes (e.g., “descend and close the gripper”), but the emitted action vector points in the wrong axis-direction. The judge catches each contradiction with phrasing like “the action’s negative-y component moves away from the target”.

When it occurs. The sign-flip rate per (source, model, variant):

- Synthetic, Opus 4.7, ACTION: 0.50 ($n=4$).
- Synthetic, Opus 4.7, ALL: 0.17 ($n=2$).
- Real, Opus 4.7, ACTION: 0.55 ($n=10$).
- Real, Sonnet 4.5, ACTION: 0.70 ($n=5$).
- Real, Opus 4.7, ALL: 0.39 ($n=9$).

Hypothesized cause. VLMs have known difficulty grounding metric 3-D axis directions (“which way is +y”) from 2-D rendered keyframes. The Fetch envs use an angled tabletop camera; the y-axis appears as a horizontal in-plane axis with no visual anchor, and the VLM frequently confuses its sign. The ALL variant reduces this rate by forcing the VLM to co-emit a position field (which it tends to get right) alongside the action, providing implicit cross-checking.

Recommended sign-check gate. The recommended mitigation for any pipeline consuming ACTION outputs is the simple gate

$$\text{accept iff } \text{sign}(a_{\text{cf}}[3]) = \text{sign}(g - \text{ag}_K) \quad \text{component-wise on axes with } |g - \text{ag}_K| > 0.01 \text{ m.} \quad (10)$$

Empirically this filters $\approx 50\%$ of ACTION outputs. The remaining $\sim 50\%$ have correct signs and are forwarded to either the verified-CF mechanism or direct action-relabeling.

Why the verified-CF mechanism subsumes this. The simulator-fork verification (Algorithm 7) is *strictly stronger* than the sign-check gate: an a_{cf} with a flipped sign on any axis points the gripper away from the goal, so the verifier env rolls out for 50 steps without firing the sparse reward, and the counterfactual is rejected as `goal_not_reached`. The sign-check gate is therefore mostly a diagnostic and a pre-filter to save the verifier’s 17.6 ms per call; on the production pipeline the verifier alone suffices.

H Reproducibility commands

For users wishing to reproduce specific results:

Headline ablation (Figure 2).

```
# Local single-seed smoke
python train.py --config configs/her_semantic_per.yaml \
  --env.name FetchPickAndPlace-v4 --training.seed 42 \
  --training.total_steps 50000

# Full Modal sweep (27 runs)
modal run modal_app.py::run_ablation
```

Counterfactual prompt evaluation (Tables 3).

```
# Synthetic (6 episodes)
python scripts/test_counterfactual.py \
  --provider openai --model gpt-4o \
  --judge_model claude-opus-4-7 \
  --variants narrative action achieved_goal all \
  --n_episodes 6 --budget 60
```

```
# Real (10 episodes)
python scripts/run_c1v2_real_data_eval.py \
  --n_episodes 10 --providers openai anthropic
```

Cross-task transfer (Figure 4).

```
python scripts/n2_validate_cross_task.py \
  --envs FetchPush-v4 FetchPickAndPlace-v4 FetchSlide-v4 \
  --n_per_env 4 --K 5
```

Verified-CF smoke (Figure 1).

```
python scripts/n1_smoke_verified_cf.py
# 4/4 expected pass; results in agent_reports/N1_smoke_results.json
```